

GUARDED CONTINGENCY SKELETONS FOR LONG-HORIZON ROBOT TASKS

Anonymous authors

Paper under double-blind review

ABSTRACT

Long-horizon task-and-motion planning (TAMP) systems often search for one symbolic skeleton and then ask continuous planners, samplers, or execution monitors to make it work. This is brittle when the correct high-level skeleton depends on physical branch conditions such as clearance, reachability, slip, or fixture state that are cheap to test but expensive to discover by failed execution. We propose guarded contingency skeletons: compact decision DAGs whose internal branch nodes are measurable physical guards and whose leaves are task-motion skeleton fragments with continuous feasibility certificates. The mechanism changes the central planning object from a single linear skeleton to a guard-indexed family of shared skeletons. We formalize guard-deterministic TAMP abstractions, prove coverage for a finite candidate set under sound and exhaustive guards, and show an exponential representation-size separation from unshared contingency trees when each action depends on only a sparse subset of reused guards. In a synthetic embodied manipulation abstraction, guarded skeletons reduce horizon-64 expected execution cost from 143.81 for replan-on-failure to 65.26 while using 273 nodes instead of 3.82 million nodes for an unshared relevant-guard contingency tree. The evidence is intentionally modest: it supports a representation mechanism and a failure mode, not real-robot readiness.

1 INTRODUCTION

Task-and-motion planning is powerful because it lets a robot reason over symbolic task structure and continuous feasibility in one problem family (Kaelbling & Lozano-Perez, 2011; 2013; Garrett et al., 2018; 2020; Toussaint, 2015; Dantam et al., 2018). A persistent habit in the field is to commit to a task skeleton, then repair geometric failures by sampling, optimization, replanning, or execution monitoring. That habit is reasonable when failures are rare, reversible, or local. It is much less reasonable when a cheap physical test would have revealed the branch condition before an irreversible attempt.

Consider a long-horizon mobile manipulation task with narrow passages, objects with uncertain grasp stability, and fixtures whose actual state is known only after contact or inspection. A linear skeleton might choose the left passage, a top grasp, and a press-fit assembly order. If any choice fails, the system can replan. But the failure may consume time, perturb the world, break a part, or force a return to an earlier checkpoint. A full contingent policy over all hidden physical regimes is also unattractive: it enumerates branches that share most of their future work.

This paper asks whether the skeleton itself can carry the contingency. We introduce guarded contingency skeletons (GCS), a representation that compiles candidate task-motion skeletons into a decision DAG. Branch nodes are physical guard predicates with safe tests, such as “clearance above threshold” or “tactile slip below threshold.” Action nodes are parameterized task-motion fragments. The DAG shares common prefixes and suffixes across guard-equivalent futures and places a guard at the first point where its value is needed for a physical certificate.

The contribution is deliberately narrow.

- We define guard-deterministic TAMP abstractions and guarded contingency skeletons.
- We give two formal claims: finite-candidate coverage under sound/exhaustive guards, and compactness relative to unshared contingency trees under sparse guard dependence.

- We provide a runnable synthetic embodied abstraction that exposes the broken assumption: late failure repair becomes expensive while full contingency enumeration explodes.
- We map the novelty boundary against sampling-based TAMP, belief-space TAMP, contingent planning, and behavior trees.

We do not claim a new complete TAMP solver, a learned perception system, real-robot validation, or a POMDP-optimal policy. The paper is about where commitment lives.

2 RELATED WORK AND NOVELTY BOUNDARY

Task-and-motion planning. Classical motion planning provides continuous feasibility machinery (Kavraki et al., 1996; LaValle, 2006; Karaman & Frazzoli, 2011). TAMP integrates that machinery with symbolic task search. Hierarchical regression and belief-space TAMP reason about manipulation goals and information-gathering actions (Kaelbling & Lozano-Perez, 2011; 2013). FFRob and PDDLStream couple symbolic search to black-box samplers and optimistic stream evaluation (Garrett et al., 2018; 2020); STRIPStream provides a related stream abstraction (Garrett et al., 2017). Logic-geometric programming optimizes trajectories under constraints induced by symbolic action sequences (Toussaint, 2015); constraint-based and incremental formulations explore similar hybrid structure (Dantam et al., 2018). These systems make linear skeleton search, sampled feasibility, and optimization over mode sequences much less novel. GCS instead asks for a compact object containing multiple skeletons indexed by physical guards.

Contingent and belief-space planning. Contingent and conformant planning produce conditional plans over symbolic observations or nondeterministic effects (Cimatti et al., 2003; Hoffmann & Brafman, 2005; Bonet & Geffner, 2000). POMDP and belief-space approaches model uncertainty probabilistically and optimize information-gathering or expected utility (Kaelbling et al., 1998; Kaelbling & Lozano-Perez, 2013). GCS does not compete with those general models. Its narrower claim is that many embodied branch variables are deterministic latent physical regimes that can be tested by guards and shared at the skeleton level without requiring a calibrated belief policy.

Behavior trees and execution monitoring. Behavior trees provide modular reactive control with sequence, fallback, and condition nodes (Colledanchise & Ogren, 2018; Iovino et al., 2022). Execution monitoring and replanning repair failures during deployment. GCS borrows the idea that conditions should affect execution, but its synthesis target is different: branch placement and sharing are derived from candidate task-motion skeleton dependencies, not manually authored fallback order.

3 GUARDED CONTINGENCY SKELETONS

3.1 GUARD-DETERMINISTIC TAMP ABSTRACTION

Let $\mathcal{G} = \{g_1, \dots, g_m\}$ be binary physical guards. A guard is a measurable predicate with a safe test action that returns its value before a class of task actions is attempted. In practice a guard could stand for a clearance interval, a reachable grasp family, a tactile slip threshold, or a fixture state. This paper assumes the guard set is given; discovering guards from raw perception is outside the claim.

A task-motion skeleton $s = (a_1, \dots, a_H)$ is a sequence of parameterized high-level actions. Each action a_t has a continuous certificate whose validity depends on a subset $D_t \subseteq \mathcal{G}$. In a guard-deterministic abstraction, for every full guard assignment $z \in \{0, 1\}^m$, the success of a certified action variant is deterministic. A candidate skeleton s_i has a guard region $R_i \subseteq \{0, 1\}^m$ for which all its certificates are valid.

Definition 1. A guarded contingency skeleton is a finite directed acyclic graph with three node types: guard-test nodes, action-fragment nodes, and terminal nodes. Each root-to-terminal path induces a task-motion skeleton. Every guard-test node queries one g_j and dispatches on its observed value. Action-fragment nodes execute certified task-motion fragments whose dependencies have already been tested on that path.

The construction used in our experiments is simple. Given candidate fragments and their guard dependencies, place each guard test immediately before the first action whose certificate depends on that guard, then hash-cons identical future subgraphs so suffixes are shared. A dominance pass removes an action alternative if its guard region is a subset of another alternative with the same suffix and no lower cost. This is not a complete planner; it is a compilation target for candidate skeletons.

3.2 COVERAGE AND COMPACTNESS

Proposition 1 (Finite-candidate coverage). *Assume (i) the candidate skeleton set S covers every guard assignment, so for each z there exists s_i with $z \in R_i$; (ii) every guard test is sound; and (iii) every action certificate for s_i is valid for all $z \in R_i$. Then executing the corresponding GCS reaches the goal for every guard assignment.*

Proof. Fix any assignment z . Sound guard tests route execution along the unique branch consistent with z . By coverage, the terminal skeleton fragment on that branch corresponds to some s_i whose region contains z . By assumption, each certificate on that skeleton is valid for z , so every action succeeds. Induction over the ordered action fragments gives goal reachability after the final fragment. $\square$

Proposition 2 (Sparse-guard representation separation). *Consider horizon H , r relevant guards, and actions whose certificates each depend on at most d guards. A direct unshared contingency tree over relevant guards stores $\Omega(H2^r)$ action occurrences. A GCS that stores one action variant per local guard signature stores $O(r + H2^d)$ guard/action nodes before dominance pruning. For constant d and growing r , the separation is exponential.*

Proof. The unshared tree has one leaf for each complete assignment to the r relevant guards, and each leaf stores a length- H skeleton, giving $\Omega(H2^r)$. The GCS stores at most r guard tests and, for each of H action positions, at most 2^d variants because only the local dependency set matters. Therefore its size is $O(r + H2^d)$. If d is constant while r grows, the ratio is exponential in r . $\square$

Both propositions are conditional. If a domain lacks safe guards, if relevant dependencies are dense, or if the candidate skeletons do not cover the true assignment, GCS provides no guarantee.

4 SYNTHETIC EMBODIED EVIDENCE

Domain. We implement a long-horizon manipulation abstraction with hidden binary physical guards. Each step depends on one or two guards, standing in for physical regimes such as clearance or grasp stability. A default linear skeleton chooses the zero-valued action variant. A failed action incurs a penalty that is larger for irreversible steps. The replan-on-failure baseline observes the relevant guards only after the failed attempt. A full contingency tree branches over all guards. A relevant unshared tree branches only over guards used by the task, but repeats the remaining skeleton at each leaf. GCS probes a guard before its first dependent action and shares the rest of the skeleton.

Protocol. We sweep horizons $H \in \{8, 16, 32, 48, 64\}$, reuse biases in $\{0.25, 0.55, 0.85\}$, 30 random seeds, and 200 sampled guard assignments per seed. The central reported setting uses reuse bias 0.55, where guard variables recur across the horizon but actions remain local. The code is in `src/guarded_skeletons.py` and `experiments/run_guarded_skeletons.py`.

Results. Figure 1 and Table 1 show the intended failure mode. At horizon 64, GCS has expected cost 65.26 and no failed actions because it tests relevant guards before commitment. Replan-on-failure has expected cost 143.81 and 16.16 failed actions per episode. The relevant unshared tree matches GCS execution cost, because it also knows the relevant guards, but it stores 3,820,748 nodes versus 273 for GCS. This supports the paper’s limited mechanism claim: when guard dependence is sparse and repeated, a skeleton DAG can keep the useful contingency without enumerating complete assignments.

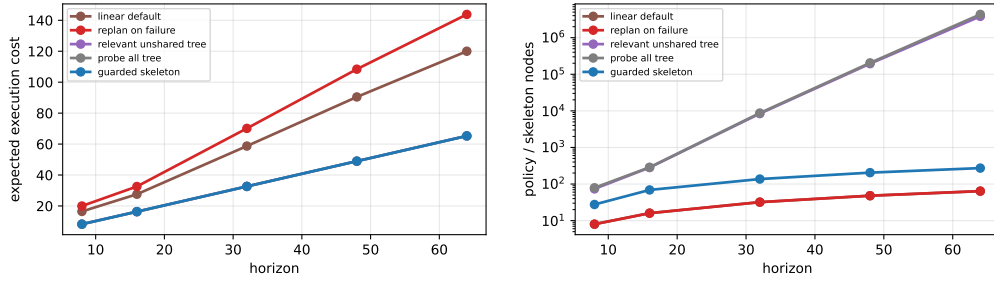


Figure 1: Synthetic guard-deterministic TAMP abstraction at reuse bias 0.55. Left: GCS pays small probe costs and avoids failed execution, while linear and replan baselines pay increasing failure penalties. Right: GCS grows near-linearly in this sparse-dependency regime, while unshared contingency trees grow exponentially in relevant guards.

Table 1: Mid-reuse setting. Nodes are policy/skeleton representation nodes, not search expansions.

Horizon	Method	Cost	Failures	Nodes
16	linear default	27.58	2.50	16
16	replan on failure	32.59	2.50	16
16	relevant unshared tree	16.32	0.00	282
16	probe all tree	16.32	0.00	287
16	guarded skeleton	16.32	0.00	69
32	linear default	58.72	5.71	32
32	replan on failure	70.14	5.71	32
32	relevant unshared tree	32.63	0.00	8413
32	probe all tree	32.64	0.00	8703
32	guarded skeleton	32.63	0.00	137
64	linear default	120.00	11.90	64
64	replan on failure	143.81	11.90	64
64	relevant unshared tree	65.26	0.00	3820748
64	probe all tree	65.28	0.00	4325375
64	guarded skeleton	65.26	0.00	273

5 DISCUSSION

Why the broken assumption matters. The literature sweep found a recurring hidden assumption: a linear skeleton can be repaired by more sampling, optimization, or replanning. In domains with irreversible actions, that assumption is false because the observation arrives after damage or lost time. The synthetic domain isolates this issue by separating probe cost from failed-action cost. When failed attempts are cheap and reversible, the advantage of GCS shrinks; when failed attempts are expensive, early guards matter.

What is genuinely new. GCS is not the first conditional plan, not the first behavior tree, and not the first TAMP representation. The novelty is the compilation unit: continuous physical feasibility dependencies become branch guards inside a compact skeleton DAG. This gives a representation-level middle ground between one linear skeleton and a full belief/contingency policy.

Limitations. The current implementation assumes guards are binary, known, safe to test, and deterministic. Real physical systems need guard discovery, multi-valued and continuous tests, noisy observations, and integration with actual TAMP solvers. The experiment is synthetic and should be read as evidence for a mechanism and a pressure point, not as a benchmark result.

6 CONCLUSION

Guarded contingency skeletons make physical branch conditions part of the task-motion skeleton rather than a late execution exception. Under explicit guard-deterministic assumptions, they provide finite-candidate coverage and can be exponentially more compact than unshared contingency trees in sparse-dependency regimes. The runnable evidence shows why this representation can matter for long-horizon embodied tasks: the cost of discovering a branch by failure compounds with horizon, while the compact guard DAG preserves shared structure.

REPRODUCIBILITY STATEMENT

The repository contains the literature matrix, analysis documents, source code, experiment script, generated CSV summaries, figures, and paper source. Reproduce the evidence with:

```
python experiments/run_guarded_skeletons.py
```

Rebuild the paper from the repository root with:

```
powershell -ExecutionPolicy Bypass -File scripts/build_paper.ps1
```

The ICLR style files were downloaded from the official ICLR Master-Template repository for ICLR 2026.

REFERENCES

- Blai Bonet and Hector Geffner. Planning with incomplete information as heuristic search in belief space. *Artificial Intelligence*, 112(1-2):1–55, 2000.
- Alessandro Cimatti, Marco Roveri, and Piergiorgio Bertoli. Conformant planning via symbolic model checking and heuristic search. In *Artificial Intelligence Planning and Scheduling*, 2003.
- Michele Colledanchise and Petter Ogren. *Behavior Trees in Robotics and AI: An Introduction*. CRC Press, 2018.
- Neil T. Dantam, Zachary K. Kingston, Swarat Chaudhuri, and Lydia E. Kavraki. An incremental constraint-based framework for task and motion planning. *The International Journal of Robotics Research*, 37(10):1134–1151, 2018.
- Caelan Reed Garrett, Tomas Lozano-Perez, and Leslie Pack Kaelbling. Stripstream: Integrating symbolic planners and blackbox samplers. In *Workshop on the Algorithmic Foundations of Robotics*, 2017.
- Caelan Reed Garrett, Tomas Lozano-Perez, and Leslie Pack Kaelbling. Ffrob: Leveraging symbolic planning for efficient task and motion planning. *The International Journal of Robotics Research*, 37(1):104–136, 2018.
- Caelan Reed Garrett, Tomas Lozano-Perez, and Leslie Pack Kaelbling. Pddlstream: Integrating symbolic planners and blackbox samplers via optimistic adaptive planning. In *International Conference on Automated Planning and Scheduling*, 2020.
- Jorg Hoffmann and Ronen I. Brafman. Contingent planning via heuristic forward search with implicit belief states. *Artificial Intelligence*, 170(6-7):503–559, 2005.
- Matteo Iovino, Edvards Scukins, Jonathan Styrud, Petter Ogren, and Christian Smith. A survey of behavior trees in robotics and ai. *Robotics and Autonomous Systems*, 154:104096, 2022.
- Leslie Pack Kaelbling and Tomas Lozano-Perez. Hierarchical task and motion planning in the now. In *IEEE International Conference on Robotics and Automation*, 2011.
- Leslie Pack Kaelbling and Tomas Lozano-Perez. Integrated task and motion planning in belief space. *The International Journal of Robotics Research*, 32(9-10):1194–1227, 2013.

Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1-2):99–134, 1998.

Sertac Karaman and Emilio Frazzoli. Sampling-based algorithms for optimal motion planning. *The International Journal of Robotics Research*, 30(7):846–894, 2011.

Lydia E. Kavraki, Petr Svestka, Jean-Claude Latombe, and Mark H. Overmars. Probabilistic roadmaps for path planning in high-dimensional configuration spaces. *IEEE Transactions on Robotics and Automation*, 12(4):566–580, 1996.

Steven M. LaValle. *Planning Algorithms*. Cambridge University Press, 2006.

Marc Toussaint. Logic-geometric programming: An optimization-based approach to combined task and motion planning. In *International Joint Conference on Artificial Intelligence*, 2015.

A ADDITIONAL EXPERIMENT DETAIL

The experiment reports aggregate rows in `results/summary.csv`. The hidden guard probability for each generated problem is sampled uniformly in $[0.25, 0.75]$. The probe cost is 0.08, the action cost is 1.0, the failed-action penalty is 4.0, and irreversible steps multiply the failure penalty by 1.5. These constants are not tuned for a particular baseline; they make the qualitative cost of late failure visible.

B AUDIT OF FORMAL SCOPE

The propositions do not prove completeness of candidate generation. They also do not address noisy guard tests or continuous guard thresholds directly. Their role is to check that the representation claim is internally consistent: if the candidate set and guards cover the world, dispatch through the GCS preserves that coverage; if local guard dependence is sparse, the shared DAG can be much smaller than a tree.